Case Study Report on

UI INTEGRATION WITH BUTTON

project submitted in fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN CSE (AI & ML)

Submitted by

B.ABHIRAM SAI	A23126552200
B.NIREESHA	A23126552201
ROHITH CHINTA	A23126552202
D.SUCHARITHA	A23126552203
D.USHA JYOTHI	A23126552204
G.KRISHNA MANAS	A23126552205
G.VAMSI KRISHNA	A23126552206
G.BHANU PRAKASH NAIDU	A23126552207

Under the Guidance of

Mr. P. Santosh Kumar

(Assistant Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A+' Grade)

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162

2025 – 2026

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A+' Grade)

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162

BONAFIDE CERTIFICATE

This is to certify that the case study entitled "**UI INTEGRATION WITH BUTTON**" submitted by **B. Abhiram Sai, B. Nireesha, Rohith Chinta, D. Sucharitha, D. Usha Jyothi, G. Krishna Manas, G. Vamsi Krishna, G. Banu Prakash Naidu** in fulfillment of the requirements for the award of Bachelor of Technology in CSE (AI & ML) in the department of Computer Science and Engineering (AI & ML) of Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam is a record of Bonafide work carried out under my guidance and supervision.

Mr. P. Santosh Kumar

ASSISTANT PROFESSOR

Department of CSE (AI & ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply modern Android UI development concepts using Jetpack Compose, Kotlin, and Material3 components in a real-world mobile development environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering (AI & ML) at ANITS, and our guide Mr. Santosh Kumar P, Assistant Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

B.ABHIRAM SAI A23126552200

B.NIREESHA A23126552201

ROHITH CHINTA A23126552202

D.SUCHARITHA A23126552203

D.USHA JYOTHI A23126552204

G.KRISHNA MANAS A23126552205

G.VAMSI KRISHNA A23126552206

G.BANU PRAKASH NAIDU A23126552207

CONTENTS

Sl. No.	Title	Page No.
1	Abstract	5
2	Introduction	5
3	Problem Statement	6
4	Aim	6
5	Objective	7
6	Tools Used	7
7	Architecture Diagram	7
8	Procedure	8
9	Implementation	9
10	Output	11
11	Conclusion	12
12	Advantages	13
13	Limitations	13
14	References	13

ABSTRACT

This project presents an Android application developed using Jetpack Compose and Kotlin that demonstrates basic UI interaction through a button component. The application features a centrally placed Material3 Button that, when clicked, triggers a Toast notification displaying the message "Button Clicked!". The entire UI is constructed using Jetpack Compose's declarative approach, eliminating the need for XML layouts.

The application is built inside a Scaffold with edge-to-edge support enabled, applying proper padding through Modifier. The InteractiveScreen composable arranges the button at the centre of the screen using Column with Arrangement.Center and Alignment.CenterHorizontally. A LocalContext is used to obtain the Android context required for displaying the Toast message.

This project highlights how Jetpack Compose simplifies Android UI development by reducing boilerplate code and enabling clean, readable component-based design. It serves as a foundational example of event handling, composable functions, and Material Design integration in a modern Android application.

INTRODUCTION

Android application development has significantly evolved with the introduction of Jetpack Compose, Google’s modern declarative UI toolkit. Traditional Android development relied heavily on XML-based layouts and View classes, whereas Jetpack Compose allows developers to describe the UI entirely in Kotlin using composable functions. This makes code more concise, easier to maintain, and aligned with modern software development practices.

This project focuses on building an interactive Android application that demonstrates one of the most fundamental aspects of mobile user interfaces: button-based interaction. The application provides a single "Click Me" button placed at the centre of the screen. When the user taps the button, a Toast message appears at the bottom of the screen displaying "Button Clicked!", giving immediate visual feedback.

Developed using Kotlin and Jetpack Compose in Android Studio, the application leverages Material3 components for consistent styling and theming. The project introduces key Compose concepts such as composable functions, modifiers, stateless UI design, and context-based system APIs like Toast. It also demonstrates how edge-to-edge display is handled using enableEdgeToEdge() and how Scaffold is used for layout structure.

This case study covers the complete development lifecycle from problem definition to implementation and output verification, providing a practical reference for students beginning their journey in modern Android development.

PROBLEM STATEMENT

Task	UI Interaction with Button
Description	Design a screen with a Button that shows a Toast on click
CO	CO2
BL (Bloom's Level)	L3 – Apply
Type	Interactive UI Task
Industry Relevance	Medium

The task requires the student to apply knowledge of Jetpack Compose composable functions, event-handling via lambda callbacks, and Android's Toast API to build a functional interactive screen. At Bloom's Level 3 (Apply), the task assesses the ability to implement learned concepts in a working application rather than merely recalling or understanding them.

AIM

To design and develop an Android application using Jetpack Compose and Kotlin that presents an interactive screen with a Material3 Button, which on being clicked displays a Toast notification, demonstrating basic event handling and UI interaction in a modern declarative Android framework.

OBJECTIVE

The primary objectives of this case study are:

1. To create an Android project using Jetpack Compose and Kotlin in Android Studio.
2. To design a centered interactive screen using Column layout with Arrangement.Center.
3. To implement a Material3 Button composable with an onClick event handler.
4. To display a Toast message using Android's Toast API upon button click.
5. To use LocalContext to access the Android application context inside a composable.
6. To apply edge-to-edge display support using enableEdgeToEdge() and Scaffold padding.
7. To demonstrate a complete Composable function with @Preview support.

TOOLS USED

Tool / Technology	Description
Android Studio	Integrated Development Environment used to build and run the Android application
Kotlin	Used for writing composable UI functions and handling user interactions
Jetpack Compose	Modern declarative UI toolkit used to build the interactive screen and Button composable
Material3	Provides styled UI components including Button, Scaffold, and Text composables
Android SDK 34	Platform used to develop and test the application on Android devices

ARCHITECTURE DIAGRAM

Architecture Diagram

User

↓

MainActivity (ComponentActivity)

↓ setContent { ... }

UIIntegrationWithButtonTheme

↓

Scaffold (Edge-to-Edge Layout)

↓ innerPadding → InteractiveScreen

Column (Arrangement.Center + Alignment.CenterHorizontally)

↓

Button ("Click Me")

↓ onClick lambda

Toast.makeText(context, "Button Clicked!", LENGTH_SHORT).show()

Architecture Explanation

The application follows the standard Jetpack Compose activity structure. The MainActivity sets the content using setContent{} inside which the Material3 theme wraps a Scaffold for edge-to-edge layout support. The Scaffold passes the innerPadding modifier down to the InteractiveScreen composable. Inside InteractiveScreen, a Column composable centres the Button both vertically and horizontally. When the user taps the button, the onClick lambda fires a Toast message via Android's Toast API using the LocalContext. The @Preview composable allows the UI to be previewed directly inside Android Studio without running the app on a device.

PROCEDURE

- Install and open Android Studio on your system.
- Create a new Android project using the Empty Activity template with Kotlin as the programming language.
- Select Minimum SDK API 24 and ensure Kotlin DSL is chosen for Gradle scripts.
- Open MainActivity.kt and replace the default content with the Jetpack Compose setup.
- Enable edge-to-edge display by calling enableEdgeToEdge() inside onCreate().
- Wrap the composable content inside UIIntegrationWithButtonTheme{} inside setContent{}.
- Add a Scaffold composable with Modifier.fillMaxSize() and pass innerPadding to InteractiveScreen.
- Create the InteractiveScreen composable function using Column with Arrangement.Center and Alignment.CenterHorizontally.
- Obtain the context using val context = LocalContext.current inside the composable.
- Add a Material3 Button with an onClick lambda that calls Toast.makeText(...).show().
- Add a Text composable inside the Button displaying "Click Me".
- Add the @Preview composable function InteractiveScreenPreview() to preview the UI.
- Run the application on a Pixel 9a emulator (API 36) or physical device and verify the Toast appears on button click.
- Push the project to GitHub at https://github.com/vamsi4956/UI-Integration-with-Button.

IMPLEMENTATION

The UI Integration with Button application was implemented in Android Studio using Kotlin and Jetpack Compose. The project contains a single MainActivity with one composable screen that renders a centered button. Clicking the button shows a Toast notification.

1. MainActivity.kt (Kotlin + Jetpack Compose)

The full source code of MainActivity.kt is provided below:

```
package com.example.uiintegrationwithbutton

import android.os.Bundle
import android.widget.Toast
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
```

```
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.tooling.preview.Preview
import com.example.uiintegrationwithbutton.ui.theme.UIIntegrationWithButtonTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            UIIntegrationWithButtonTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    InteractiveScreen(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun InteractiveScreen(modifier: Modifier = Modifier) {
    val context = LocalContext.current
    Column(
        modifier = modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Button(onClick = {
            Toast.makeText(context, "Button Clicked!", Toast.LENGTH_SHORT).show()
        }) {
            Text(text = "Click Me")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun InteractiveScreenPreview() {
    UIIntegrationWithButtonTheme {
        InteractiveScreen()
    }
}
```

2. Explanation of Code

MainActivity extends ComponentActivity and calls enableEdgeToEdge() to allow the app to draw behind system bars. Inside onCreate(), setContent{} renders the Compose UI tree.

UIIntegrationWithButtonTheme wraps all composables in a Material3 theme, ensuring consistent colours, typography, and component styles.

Scaffold provides the top-level layout structure and supplies innerPadding to avoid content being obscured by system insets.

InteractiveScreen is a @Composable function that uses Column with verticalArrangement = Arrangement.Center and horizontalAlignment = Alignment.CenterHorizontally to place the button exactly in the middle of the screen.

LocalContext.current provides the Android Context needed to call Toast.makeText() inside the composable without passing it as a parameter.

Button(onClick = {...}) is a Material3 Button composable. The onClick lambda calls Toast.makeText(context, "Button Clicked!", Toast.LENGTH_SHORT).show() to display a brief notification when tapped.

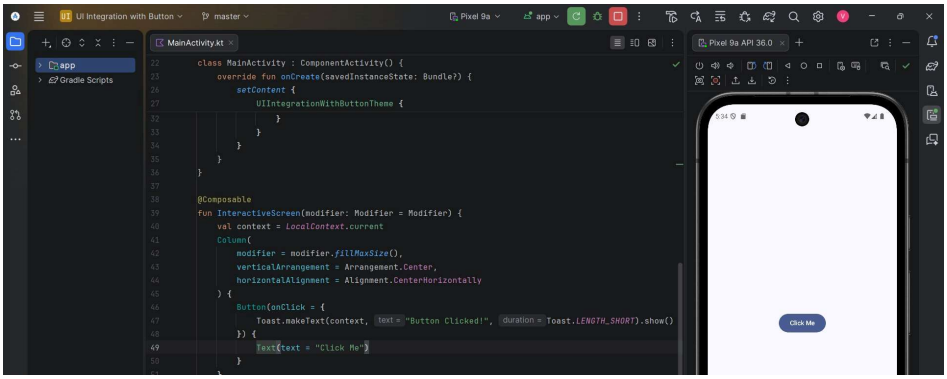
@Preview annotates InteractiveScreenPreview() which wraps InteractiveScreen() in the theme, enabling live preview inside Android Studio's Design panel without deploying to a device.

OUTPUT

After running the application on a Pixel 9a API 36 emulator, the following screens were observed:

- The application launches and displays a white screen with a single "Click Me" button placed at the centre of the screen.
- When the user taps the "Click Me" button, a Toast notification appears at the bottom of the screen displaying "Button Clicked!".
- The Toast disappears automatically after a short duration (LENGTH_SHORT ≈ 2 seconds).

Screenshot 1 – Initial Screen (Button Before Click)



The screen shows the app running on the Pixel 9a emulator at 5:34. The "Click Me" button is visible at the centre of the screen with a dark blue Material3 style. The Jetpack Compose source code is visible in Android Studio on the left.

Screenshot 2 – Toast Notification After Button Click

After tapping the "Click Me" button, the Toast message "Button Clicked!" appears at the bottom of the screen. The Android panel on the left shows the build was successful (green run button).

GitHub Repository

The complete source code is available at: **https://github.com/vamsi4956/UI-Integration-with-Button**

RESULTS

The Android application was successfully built and executed on a Pixel 9a API 36 emulator. The "Click Me" button renders correctly at the centre of the screen using Jetpack Compose's declarative layout system. On button tap, the Toast notification fires immediately and disappears after the short duration. The @Preview annotation confirmed correct visual layout inside Android Studio's Design panel. All project objectives were achieved and the application functions as expected per the problem statement requirements.

CONCLUSION

This project successfully demonstrates the implementation of basic UI interaction in an Android application using Jetpack Compose, Kotlin, and Material3 components. The application achieves its primary goal of displaying a Toast notification when a button is clicked, addressing the Apply level of Bloom's Taxonomy (BL: L3) as specified in the problem statement.

The use of Jetpack Compose significantly reduces boilerplate compared to traditional XML-based Android development. Composable functions, lambda-based event handling, and LocalContext make the code concise and readable. The Scaffold and edge-to-edge support ensure the application adheres to modern Android UI guidelines.

The project provides a foundational understanding of how interactive components work in Compose-based Android apps and serves as a building block for more complex UI interactions in future assignments.

ADVANTAGES

- Uses Jetpack Compose's declarative approach, reducing UI code significantly compared to XML-based development
- No XML layout files required — entire UI is written in Kotlin
- Material3 Button provides consistent styling and theming out of the box
- LocalContext eliminates the need to pass Android Context manually through parameters
- @Preview support enables rapid UI iteration inside Android Studio without a device
- Edge-to-edge display with Scaffold ensures compatibility with modern Android devices
- Simple and lightweight application with fast startup time
- Directly maps to industry-standard modern Android development practices

LIMITATIONS

- Limited to a single screen with no navigation or multi-screen support
- Toast notifications are deprecated in some Android versions and may not appear uniformly on all devices

- No persistent state management — button click count or events are not stored
- Does not demonstrate advanced Compose state management (remember, mutableStateOf)
- Runs only on Android devices — no cross-platform support
- No backend integration or data persistence
- Limited accessibility support (no content descriptions on the button icon)
- Depends on device performance for smooth rendering of animations

REFERENCES

8. Android Developers. Jetpack Compose Documentation. Available: https://developer.android.com/jetpack/compose
9. JetBrains. Kotlin Programming Language Documentation. Available: https://kotlinlang.org
10. Google. Material Design 3 Guidelines. Available: https://m3.material.io
11. Android Developers. Toast API Reference. Available: https://developer.android.com/reference/android/widget/Toast
12. GitHub. UI Integration with Button Repository. Available: https://github.com/vamsi4956/UI-Integration-with-Button
13. Android Developers. Activity and Lifecycle Documentation. Available: https://developer.android.com/guide/components/activities/activity-lifecycle